ADR-001 · The Golden Path DevEx Platform

Status: Proposed · **Date:** 2026-07-24 · **Author:** Vinicius · **Deciders:** DevEx Platform · Internal Tools as a Product

CONTEXT

LoanPro runs 10+ independent, full-cycle engineering teams across multiple languages (Python, Go, Clojure, TypeScript). Left to themselves, each team invents its own git conventions, pipelines, and metrics — so DORA numbers are **not comparable** and SOC 2 auditing is inconsistent. We treat Developer Experience as a **product**: a "Golden Path" that homologates the lifecycle while remaining low-friction and non-blocking, so teams adopt it because it is the easiest route, not because it is mandated.

DECISION

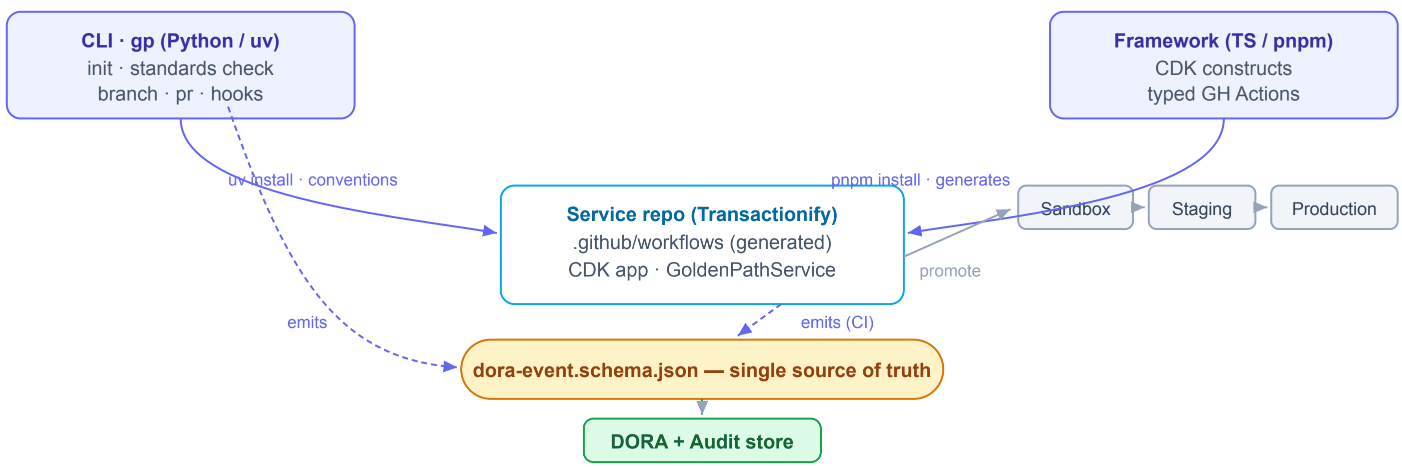
Ship a two-component ecosystem, each independently distributed **from Git** (no registry publish), wired together by a single **language-agnostic event contract**.

Component A — CLI (Python, `uv`)
The developer interface. Standardizes local workflows (`init`, `standards check`, branch/PR conventions, git hooks) and enforces the **Universal Work ID**. Emits audit/DORA events.

Component B — Framework (TypeScript, `pnpm`)
Shared **AWS CDK constructs** + **type-safe GitHub Actions** generation (built on `github-actions-workflow-ts`). Produces the Stack-Aware PR pipeline and captures DORA in CI.

The contract — `dora-event.schema.json` — is the single source of truth. Both components emit the identical event shape; language is metadata only. *This is what makes DORA comparable between a Python team and a Go team*, and it is defined **before** either tool.

ARCHITECTURE



PIPELINES

PR Pipeline — implemented

- **Small Tests:** unit + Property-Based Testing + API-contract validation (no deploy).
- **Deployment:** promote Sandbox → Staging → Production via AWS CDK.
- Two-Reviewer rule + standardized PR template; Work ID enforced in branch/commit/title.

Integration Pipeline — designed

- Triggered on merge to `main`; final validation + production deploy.
- Reuses the same stages and the same event schema, so DORA stays continuous across both pipelines.
- PoC builds only the PR pipeline; this remains a config flag on the shared generator.

HOMOLOGATION — HOW 10+ TEAMS ADOPT BOTH TOOLS

- **Convention over configuration:** `gp init` scaffolds correct-by-default; the golden path is the path of least resistance.
- **One-command install from Git:** `uv tool install git+…` and `pnpm add git+…`, versioned by tag — nothing to publish or operate.
- **Dogfooding + a reference service:** the platform repo and the Transactionify fork run the exact pipeline as a copyable starting point.
- **Enforcement at the edges:** git hooks and PR-pipeline gates make non-conformance fail fast, so adoption is pulled by developers rather than pushed by mandate.

SCALABILITY — AVOIDING THE PLATFORM-TEAM BOTTLENECK

- **Inner-source model:** teams contribute via PRs against documented extension points (a new language profile ≈ 4 files, a new pipeline stage ≈ one builder function). No platform-team code required.
- **Stack-Aware & pluggable:** language support is data, not a fork; adding Go/Clojure is additive, never a framework rewrite.
- **Library, not a service:** the framework is a versioned dependency each team pins and upgrades on its own cadence — the platform team is never in the request path.

SHIFT-LEFT STRATEGY

- **Pre-push git hooks (CLI-managed):** run tests/lint before code leaves the machine.
- **Fast PR "Small Tests" stage:** unit + PBT + API-contract before any deploy.
- **Local environment parity:** run the service and its dependencies locally to execute tests without cloud latency — shrinking the defect-introduction → detection gap (the feedback-loop metric under evaluation).

CONSEQUENCES

- **+** Comparable DORA across languages; consistent SOC 2 audit trail; low-friction adoption.
- **+** Platform team scales by curation, not by doing each team's pipeline work.
- **–** Two runtimes (Python + TS) to maintain; contract changes need a coordinated release.
- **–** Git-based distribution trades registry ergonomics for zero-publish simplicity (fine for PoC).

ALTERNATIVES CONSIDERED

Option	Why rejected
Single-language toolchain	Violates polyglot future-proofing
Central CI service owned by platform	Recreates the bottleneck we must avoid
Hand-written YAML templates	No compile-time safety; drifts per team